

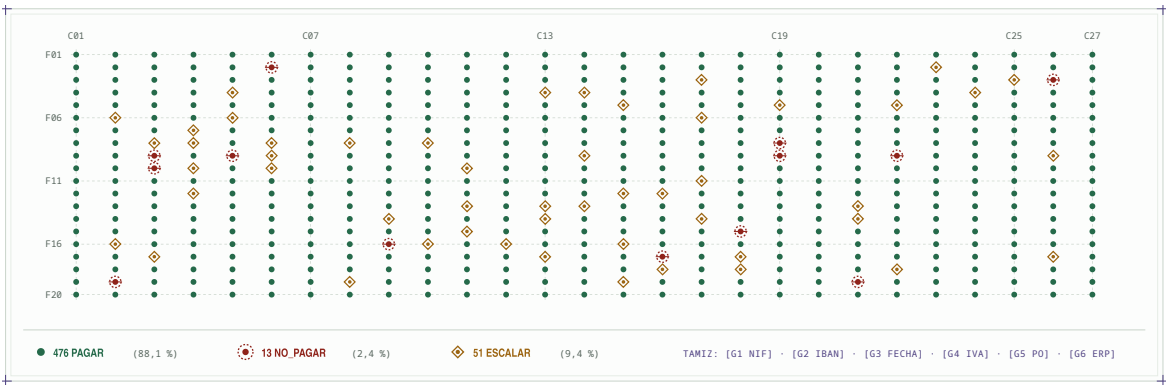
ARQUITECTURA DE REGLAS DETERMINISTAS & AUDITORÍA FORENSE

Otrebla.

Qué facturas pagar, y por qué.

FIG. 1 · MATRIZ DETERMINISTA · DOS LOTES · 540 FACTURAS

476 PAGAR · 13 NO_PAGAR · 51 ESCALAR



BALANCE ACUMULADO · LOTE 1 + LOTE 2 · 540 FACTURAS



476 88,1 %

PAGAR

2.517.117,84 €

Las seis reglas pasan limpiamente.
Se abonan de forma desatendida sin
riesgo operativo.

13 2,4 %

NO_PAGAR

21.088,70 €

Duplicidad probada frente al
ERP o dentro del lote. Capital
protegido de inmediato.

51 9,4 %

ESCALAR

1.079.045,60 €

Discrepancias en IBAN, NIF,
fecha o pedido. Expediente
aislado con el dato que no
cuadra ya señalado.

1 Alberto contra las facturas

El trabajo de Alberto es aprobar los pagos. Dicho así, parece darle a un botón. Ojalá. Por cada factura tiene que abrir el PDF, encontrar el pedido en un Excel de catorce hojas, comprobar el NIF y el IBAN del proveedor, sumar base e IVA y entrar en un ERP del 2009 que sigue vivo porque nadie se atreve a cambiarlo, solo para comprobar si ya está pagada. Con una factura es un trabajo tedioso. Con miles, Alberto empieza a sospechar que cuando su padre le dijo que estudiase algo "con salidas", se refería a salidas de emergencia.

Por suerte, nuestro sistema se encarga del trabajo pesado. Alberto sigue teniendo la última palabra, pero ya no son palabrotas. Así funciona:

476 PAGAR · 88,1 % 2.517.117,84 € Las seis reglas pasan. Se paga sin que nadie las mire.	13 NO_PAGAR · 2,4 % 21.088,70 € Pago duplicado demostrado. No es una duda: es un no.	51 ESCALAR · 9,4 % 1.079.045,60 € Hace falta una persona o un dato corregido.
--	--	---

Lo que el sistema evita

[outcomes*.jsonl](#)

Los trece **NO_PAGAR** de ambos lotes son dinero concreto: 21.088,70 €. El artefacto aparta esas facturas del circuito automático y conserva en la traza la prueba del ERP, del lote actual o del historial que impide pagarlas.

Qué queda para revisión

En el lote 1 escalan 37 facturas por 123.307,20 €. El lote 2 añade 14 expedientes y 955.738,40 € a la cola. En total son 51 facturas y 1.079.045,60 €. El JSONL conserva el veredicto contractual; la traza interna mantiene los hallazgos y la evidencia que lo explican.

El sábado dejó de ser una hipótesis

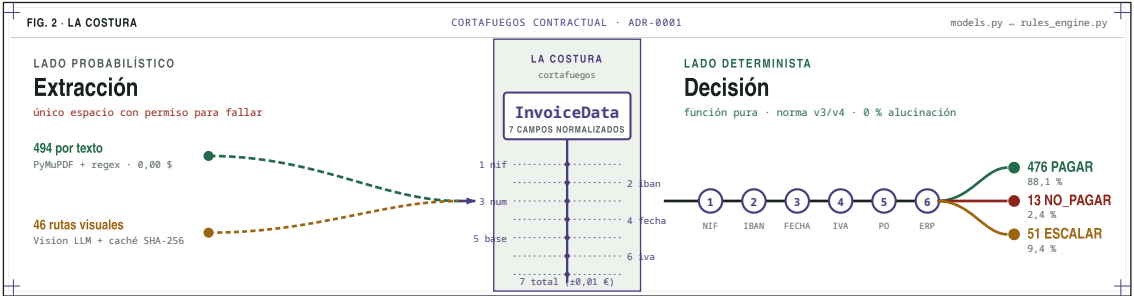
[outcomes_lote2.jsonl](#)

El segundo lote añade 40 facturas, una exportación nueva del ERP, 40 pedidos y cuatro proveedores internacionales. Llegan plantillas nuevas, idiomas distintos y 17 documentos que necesitan visión. Se ejecutan con `--batch lote2`, se contrastan también con los pedidos ya vistos y escriben un artefacto propio sin tocar `outcomes.jsonl`. El resultado: 25 **PAGAR**, 1 **NO_PAGAR** y 14 **ESCALAR**. La misma consola permite procesarlo, seguirlo y cerrar sus excepciones.

2 Arquitectura

models.py

Leer un documento es un problema probabilístico; decidir un pago no lo es. Separamos las dos cosas con un contrato de datos en medio, y sólo el lado izquierdo tiene permiso para equivocarse.



Con esto, la pregunta que importa cuando algo sale mal es: ¿leímos mal el documento o aplicamos mal la norma? Y es que son dos capas con dos arreglos distintos

2.1 Las etapas

0	Datos de referencia business_data erp_snapshot	Limpia el Excel, fusiona los CSV incrementales del sábado y baja el ERP entero. DEJA la foto versionada en SQLite, con fecha, avisos y reintentos. Lote 2 exige que la actualización conste como cargada.
1	Ingesta pipeline._collect_files	Lista la carpeta y descarta repetidos por nombre y por hash del contenido. DEJA la lista del lote con el SHA-256 de cada archivo.
2	Extracción HybridExtractor	Ocho hilos y un extractor aislado por hilo: texto local; visión para escaneos o texto incompleto; línea base si falla. DEJA campos, método, latencia, coste y evidencia, conservando el orden estable de entrada.
3	Decisión rules_engine	Aplica la norma versionada y busca pedidos repetidos en este lote y en los anteriores. DEJA veredicto, motivo, todos los hallazgos, las 13 comprobaciones y su evidencia.
4	Estado y entregable state · deliverables	Guarda la decisión vigente y la histórica. Escribe el JSONL en un temporal y lo renombra: o está entero o no está. DEJA tres tablas y un artefacto aislado por lote.
5	Console web/ · Next.js	Cola de revisión, expediente de cada archivo y política editable. DEJA nada propio: lee y escribe a través de la capa de Python. Si se cae, el lote sigue funcionando solo.

Las etapas 0 a 4 siguen siendo una orden y dos SQLite. No hay cola distribuida: el paralelismo útil vive dentro de la extracción, que espera red; la decisión y la escritura permanecen deterministas. Lo que cambiaría con otro volumen está en las páginas 6 y 7.

2.2 Estado y reparto del trabajo

state.py

El estado vive en SQLite. El JSONL es el formato de entrega, no la verdad. Esa distinción permite reprocesar sin duplicar.

Bastan tres tablas. `decisions` guarda la decisión vigente de cada archivo, una por `file_id`: reprocesar la sustituye, nunca la duplica. `decision_history` guarda todas las anteriores, inmutables y con su lote, para ver qué cambió al cambiar la norma o el ERP. `runs` guarda cada ejecución: recuentos, segundos, archivos por segundo y coste. La foto del ERP vive al lado, con la fecha de cada descarga.

Código determinista	Modelo de visión	Persona
540 de 540 decisiones	46 rutas de visión	Sólo las excepciones
Lee los documentos digitales y decide las facturas de ambos lotes.	Convierte 29 escaneos del lote 1 y 17 formatos del lote 2 en campos validados.	Resuelve escalados, corrige campos con una anulación firmada y ajusta la política.
No interpreta imágenes. Si el PDF no trae texto, no lo intenta.	No decide. Su salida entra en las mismas reglas que todo lo demás.	No revisa lo que pasa las reglas. Ese es el trabajo que se ahorra.

Las correcciones humanas se declaran en un fichero atado al SHA-256 exacto del PDF. Si el documento cambia, la corrección caduca sola y el archivo vuelve a escalar. En el lote 1 hay una (`scan_021.pdf`), anotada como `+human-override`.

2.3 La norma versionada, regla a regla

rules_engine.py

La v3 vive en código como seis reglas y trece comprobaciones. Como el material del sábado no incluyó el texto oficial de v4, `norma-v4` es un alias explícito de esa lógica: queda estampado como v4 y emite un aviso; v5 o cualquier versión desconocida falla de inmediato. No hay degradación silenciosa.

Regla	Contra qué se compara	Códigos y veredicto por defecto
0 Extracción completa	Los campos necesarios para aplicar las reglas.	<code>incomplete_extraction</code> ESCALAR
1 Proveedor e IBAN	Maestro de proveedores del Excel, por NIF.	<code>supplier_not_in_master</code> , <code>iban_mismatch</code> ESCALAR
2 Pedido e importe	Hoja de pedidos: que exista, que sea de ese proveedor y que el total cuadre.	<code>pedido_not_found</code> , <code>pedido_supplier_mismatch</code> , <code>amount_mismatch</code> ESCALAR
3 Aritmética del IVA	La propia factura: base más IVA, y base por tipo.	<code>total_not_base_plus_iva</code> , <code>iva_miscalculated</code> ESCALAR
4 Fecha	La fecha de referencia de la ejecución.	<code>invalid_date</code> , <code>future_date</code> ESCALAR
5 ERP y duplicados	El asiento del ERP, el resto del lote y los lotes anteriores.	<code>already_paid</code> , <code>duplicate_pedido</code> NO PAGAR ; el resto ESCALAR

La tolerancia de importes es de un céntimo. Cuando varios hallazgos apuntan a sitios distintos, la precedencia es **ESCALAR** > **NO PAGAR** > **PAGAR**. El mapa de los 18 códigos a veredictos es un fichero de política editable desde la consola: si Alberto decide que un IBAN distinto es rechazo y no revisión, cambia configuración, no código, y el cambio queda registrado en la ejecución. Los códigos que no se disparan en este lote están cubiertos por las pruebas del motor.

2.4 Seguir una decisión

Esto es lo que el sistema guarda de [2026-0811-B_catering.pdf](#): una factura de Catering Hermanos Pico por 2.920,10 € contra un pedido de 2.795,10 €, decidida en 4,85 ms. No es un resumen escrito para este documento: son las trece comprobaciones tal como salen del motor, cada una con el valor observado, el esperado y la fuente del esperado.

R	Comprobación	Resultado	Observado	Esperado, y de dónde sale
0	Extracción completa	pasa	texto incrustado	campos suficientes Factura
1	Proveedor registrado	pasa	B96233419 Factura	B96233419 Excel · Proveedores · P005
1	IBAN del proveedor	pasa	ES1800815290070001234567 Factura	ES1800815290070001234567 Excel · Proveedores · P005
2	Pedido existente	pasa	PO-2026-0707 Factura	PO-2026-0707 Excel · Pedidos_2026
2	Proveedor del pedido	pasa	P005 Factura y maestro	P005 Excel · Pedidos_2026 · PO-2026-0707
2	Total frente al pedido	no pasa	2.920,10 € Factura	2.795,10 € Excel · Pedidos_2026 · PO-2026-0707
3	Base más IVA	no pasa	2.920,10 € Factura	2.795,10 € Cálculo: 2.310,00 + 485,10
3	Cálculo del IVA	pasa	485,10 € Factura	485,1000 € Cálculo: 2.310,00 × 21 %
4	Fecha no futura	pasa	2026-05-26 Factura	≤ 2026-09-19 Fecha de referencia de la ejecución
5	Asiento en el ERP	pasa	PO-2026-0707 Factura	PO-2026-0707 ERP · AS-70006
5	Total frente al ERP	no pasa	2.920,10 € Factura	2.795,10 € ERP · AS-70006
5	Estado de pago	pasa	PENDIENTE ERP · AS-70006	PENDIENTE Norma de pagos v3
5	Pedido único en el lote	pasa	1 aparición	1 Lote 1 y lotes anteriores

Y lo que acompaña al veredicto

[decisions](#)

Junto a las trece filas queda el veredicto con su motivo principal ([ESCALAR · amount_mismatch](#)), los tres hallazgos completos, la evidencia (P005 · PO-2026-0707 · AS-70006 · PENDIENTE), cómo se leyó el PDF (`embedded_text`, plantilla narrative), la norma aplicada (norma-v3), la latencia (4,85 ms), el coste (0,00 \$) y la hora de ejecución, 2026-09-19T11:20:35Z, con la foto del ERP de esa misma mañana.

Las señales en directo

El proceso emite una línea JSON por suceso: `run_start`, `extracted`, `decided`, `override_skipped`, `data_error`, `data_warnings` y `run_done` con el resumen. La consola consume ese mismo flujo: lo que Alberto ve en pantalla es lo que se guarda en la base de datos. No hay dos caminos.

El trabajo pendiente no es una métrica aparte: es la consulta `result = 'ESCALAR'`. En los dos artefactos devuelve 51 filas y 1.079.045,60 €.

2.5 Escala y coste

runs

Esta página separa dos cosas: lo que medimos en una ejecución real y lo que estimamos a partir de esas medidas. Lo segundo va marcado como estimación.

Medida	Valor	De dónde sale
Lote 1 completo, de carpeta a JSONL	2,055 s	Columna elapsed_s de la ejecución.
Rendimiento	243,3 arch./s	Columna files_per_s, con los 29 escaneos ya en caché.
Extracción por archivo con texto	4,1 ms	Media de 500 filas; mínimo 0,83 ms, máximo 16,64 ms.
Extracción por visión, en frío	5,74 s	Media de los 29 escaneos; mediana 5,22 s, rango de 4,36 a 8,60 s.
Descarga completa del ERP	3,489 s	516 asientos, 26 páginas, 30 peticiones, 3 reintentos.
Coste en modelos de la ejecución	0,00 \$	471 archivos no llaman a ningún modelo; los 29 restantes salieron de caché.

Todo en un portátil: Intel Core i5-12500H, 16 GB, Debian 13 y Python 3.13. Sigue siendo un solo proceso, ahora con ocho hilos para solapar la espera de red de la visión.

Coste por lote

```
coste_lote = n_escaneos_nuevos × precio_visión
471 digitales × 0 $ + 29 escaneos × 0,0003 $ ≈ 0,0087 $ por los 500 archivos
precio_visión ≈ 0,0003 $/página (0,15 $/M tokens de entrada, 0,60 $/M de salida)
```

El término que falta es el que importa: **los documentos con texto no aparecen en la fórmula**, porque nunca llaman a un modelo. El coste no crece con el número de facturas; crece con el número de *escaneos nuevos*. Y como la caché indexa por contenido, reejecutar un lote entero cuesta cero.

A 100.000 facturas

Con la misma proporción de escaneos (5,8 %), *estimamos* 5.800 llamadas de visión y unos 1,74 \$ en modelos. A esa escala el coste dominante no es el modelo: es el tiempo. El pool ya es configurable con `--workers` o `MAISA_EXTRACT_WORKERS`; se reduce ante cuotas agresivas.

Las 94.200 facturas con texto tardan unos 6,5 minutos en un solo proceso ($94.200 \div 243$). Es trabajo por archivo sin estado compartido: con 8 trabajadores baja de un minuto. Los 5.800 escaneos son el cuello de botella real: 9,2 horas en serie, 35 minutos con 16 llamadas en paralelo. El techo lo pone la cuota del proveedor, no la máquina. Decidir las 100.000 tarda menos de un minuto: es una función pura en memoria contra dos índices.

Escalar este sistema no es escalar la decisión. Es administrar la cuota de visión y repartir la extracción entre procesos. Las dos cosas viven aisladas en la etapa 2.

2.6 Lo que aguanta y lo que no

Estos son los límites del diseño actual y lo que haríamos en cada caso. Ninguno es una sorpresa: son el precio de elegir lo más pequeño que resolvía el problema.

SQLite admite un solo escritor.

Aparece con más de un proceso escribiendo. La capa de estado es un módulo aislado: se cambia por Postgres sin tocar reglas ni extractores.

El ERP tolera 10 peticiones/s y la sesión caduca a los 15 minutos o 300 usos.

Ya lo esquivamos: una foto completa en 30 peticiones, y las 500 decisiones leen de SQLite. Sin esa foto serían 500 llamadas y una sesión caducada a mitad del lote.

El Excel y los CSV incrementales se cargan enteros en memoria.

Con cientos da igual. Con millones, pasa a ser una tabla indexada del mismo almacén de estado.

La cuota del proveedor de visión es el techo del paralelismo.

Se reparte entre varios proveedores, y la caché absorbe las reejecuciones.

Añadir un tipo de archivo nuevo

El punto de extensión sigue siendo InvoiceData, pero el interruptor ya está conectado. La política habilita PDF e imágenes, FacturaE/UBL, EML/MSG y adjuntos, DOCX, XLSX/CSV, texto, HTML y JSON. Cada adaptador normaliza; un primer pase determinista intenta leerlo, un segundo pase de modelo recupera lo incompleto y, si no es factura, lo enruta a out_of_scope, unreadable o unknown_format. Nunca convierte un formato desconocido en un pago.

Idempotencia

- La clave es el file_id: reprocesar sustituye la decisión, no añade otra.
- La ingesta descarta repetidos por nombre y por hash; la visión se cachea por ese mismo hash.
- Los pedidos de lotes anteriores se leen del historial: una factura repetida entre lotes se ve.
- El JSONL se escribe en un temporal y se renombra: o está entero o no está.

Lo que cubren las pruebas

tests/ · 58 pruebas

58 pruebas en 2,41 s cubren el motor, los maestros del lote 2, el aislamiento entre artefactos, XML/formatos, política, historial, diferencias entre ejecuciones, caché y norma versionada. El motor se prueba sin abrir un PDF.

La precisión de lectura se comprobó a mano sobre 45 documentos: 98,4 % de campos correctos, 100 % en los 30 digitales y 95,2 % en los 15 escaneados. Los dos documentos con algún campo mal son escaneos degradados a propósito, y los dos acabaron escalados. Ningún error de lectura produjo un **PAGAR** indebido.

2.7 Resiliencia y recuperación

Ningún fallo puede convertirse en un pago. Cuando el sistema no sabe, escala; cuando no puede, se para. Nunca rellena el hueco con algo plausible.

El proveedor de visión no responde

Tres intentos con espera creciente (0, 1 y 2 s) y salto a los modelos alternativos. Los caminos locales y los documentos en caché no se enteran. Al relanzar, sólo se reintentan los documentos sin resolver.

Caen todos los proveedores

La extracción se marca como no disponible y la regla 0 no pasa. Esos archivos salen como **ESCALAR** y aparecen en la cola. El resto del lote se decide con normalidad. Se corta la clave en directo y esto es lo que se ve.

El modelo devuelve algo que no es una factura

La salida se valida contra el esquema y se rechaza. Si el extractor revienta, cae a la línea base y lo anota en el método. Nada contaminado entra en las reglas, y un archivo malo no interrumpe el lote: se reintenta y, si insiste, escala.

El ERP falla como suele fallar

ORA-00600: reintenta la misma llamada, hasta 6 veces. 401: vuelve a autenticarse. 429: espera lo que diga la cabecera, y de salida no pasamos de 8,3 peticiones/s sobre un techo de 10. Medido: 3 reintentos en 30 peticiones, sin intervención.

El ERP no está

El lote no arranca. Queda la foto anterior con su fecha, pero decidir con una foto vieja tiene que ser una elección, no un accidente.

Se interrumpe el proceso

Las decisiones ya guardadas y la caché de visión se conservan; el JSONL anterior queda intacto porque se escribe aparte y se renombra al final. Se relanza y los hashes evitan pagar dos veces por lo ya leído. Para medir o corregir una extracción existe --force: omite la caché; borrar un lote desde la consola también la purga.

Se pide una norma desconocida

Error inmediato. La única excepción declarada es norma-v4: ante la ausencia del texto oficial, reutiliza v3 con etiqueta v4 y aviso visible. Una v5 no cae hacia atrás.

Qué ocurrió con el lote 2 real

--batch lote2 selecciona sus 40 PDFs, norma-v4, la foto actualizada del ERP y outcomes_lote2.jsonl; el lote 1 queda intacto. Los CSV del sábado añaden 40 pedidos y los proveedores P012–P015 con NIF e IBAN no españoles, y se fusionan en cualquier ejecución de la consola para que una subida manual vea el mismo universo.

De los 40 documentos, 23 salen por texto y 17 por visión/cache. La visión ya no se reserva a PDFs sin texto: si una plantilla digital nueva o multilingüe deja campos obligatorios incompletos, se rasteriza y se adopta la lectura visual sólo si recupera más campos. Cada hilo tiene su extractor para que método, coste y evidencia no se mezclen; el resultado final recupera el orden original.

Antes de decidir se consultan los pedidos históricos de otros lotes. Después, la pantalla *Cambios* compara dos ejecuciones inmutables y enumera transiciones de veredicto, motivos, altas y bajas: adaptar norma, ERP o maestro deja un impacto verificable.

3 ADRs y trade-offs

De estas cuatro decisiones depende todo lo anterior. Cada una lleva lo que renunciamos a tener, no sólo lo que ganamos.

ADR-01 · aceptada

Separar la lectura del documento de la decisión de pago

Contexto	Las facturas llegan en seis plantillas y algunas son sólo imagen: leerlas exige modelos probabilísticos. Decidir un pago, en cambio, tiene que ser reproducible y defendible ante una norma escrita.
Alternativas	1. Un modelo lee la factura y dicta el veredicto en la misma llamada. 2. Sólo expresiones regulares, para leer y para decidir. 3. Normalizar todo a un esquema y decidir después con un motor determinista.
Decisión	La tercera. La extracción produce InvoiceData; el motor de reglas consume InvoiceData y no sabe qué es un PDF. Sólo la extracción tiene permiso para ser probabilística.
Consecuencias	Mantener un esquema canónico cuesta coordinación: cambiarlo toca extractores, persistencia y reglas a la vez. Y una lectura equivocada pero plausible sigue pudiendo arrastrar la decisión, así que la evidencia de origen y el escalado siguen siendo necesarios. Aceptamos las dos cosas.
Evidencia	El motor se prueba sin abrir un PDF. La traza de la página 5 dice, para cada comprobación, si el valor discutido viene de la factura, del Excel o del ERP. Ésa es la pregunta que esta separación deja contestar.

ADR-02 · aceptada

Escalar la duda, y reservar el «no» para lo demostrado

Contexto	Pagar una factura dudosa cuesta dinero. Rechazar toda anomalía bloquea pagos legítimos y desgasta la confianza en el sistema. La norma pide revisión humana cuando hay duda razonable.
Alternativas	1. Toda comprobación fallida es NO_PAGAR. 2. Un modelo resuelve los casos ambiguos. 3. ESCALAR para la duda, y NO_PAGAR sólo para el pago duplicado demostrable.
Decisión	La tercera, con precedencia ESCALAR > NO_PAGAR > PAGAR y un mapa de códigos a veredictos configurable. Descartamos la segunda a propósito: un modelo que adjudica casos límite es justo la parte que no podríamos defender ante un auditor.
Consecuencias	La cola de revisión es más larga de lo estrictamente necesario, y la precedencia puede devolver ESCALAR aunque otro hallazgo, por sí solo, ya diga que no se pague. Sale a cuenta si la consola enseña el desajuste exacto. Por eso la consola no es un adorno: es la contrapartida de esta decisión.
Evidencia	37 escalados frente a 11 rechazos en el lote 1. Los 11 se apoyan en un estado PAGADA del ERP o en un pedido repetido dentro del lote, no en una valoración. Y ningún error de lectura detectado en la revisión manual produjo un PAGAR indebido.

ADR-03 · aceptada

Paralelizar la espera, no la decisión

Contexto	La visión tarda segundos y espera red; las reglas tardan milisegundos, necesitan ver el lote completo para detectar duplicados y terminan en un único artefacto auditable. Procesarlo todo en serie desperdicia tiempo, pero paralelizarlo todo introduce carreras.
Alternativas	1. Una factura detrás de otra, de lectura a escritura. 2. Un trabajador independiente decide y persiste cada factura. 3. Extraer en paralelo; reunir en orden; decidir y escribir una sola vez.
Decisión	La tercera. Un <code>ThreadPoolExecutor</code> reparte la extracción entre ocho hilos, con un extractor aislado por hilo. Después se recupera el orden de entrada y el motor decide el lote completo en memoria; SQLite y el JSONL conservan un solo escritor.
Consecuencias	Solapamos la latencia de visión sin compartir método, coste ni evidencia entre facturas, y la salida sigue siendo reproducible. Aceptamos que la decisión no escale horizontalmente y que la cuota del proveedor limite el número útil de hilos.
Evidencia	Los dos lotes pasan por la misma ruta: 540 decisiones recuperan su orden estable y 46 documentos toman la vía visual. <code>--workers</code> permite reducir concurrencia ante cuotas agresivas; 58 pruebas cubren orden, aislamiento, caché y artefactos.

ADR-04 · aceptada

Guardar el estado en SQLite, y tratar el JSONL como entrega y no como verdad

Contexto	Hay que saber la decisión vigente de cada archivo, conservar las métricas de cada ejecución y reprocesar cuando cambien la norma o los datos. La consola y el proceso por lotes tienen que ver lo mismo, sin montar infraestructura para un fin de semana.
Alternativas	1. Sólo los ficheros JSONL generados. 2. Ficheros JSON sueltos, uno por archivo. 3. Una base de datos gestionada, con su despliegue. 4. SQLite local, propiedad exclusiva de la capa de Python.
Decisión	La cuarta: <code>runs</code> , <code>decisions</code> con <code>file_id</code> como clave primaria y <code>decision_history</code> inmutable. El JSONL se genera desde ahí. La consola no abre el fichero SQLite: pasa por la misma capa de Python, para que el estado tenga un solo dueño.
Consecuencias	Un solo escritor: en cuanto haya varios trabajadores hay que migrar. Lo asumimos porque la capa de estado está aislada y la migración es local. La tabla de decisiones vigentes no guarda las versiones anteriores de una factura; de ahí el historial. Y una recuperación más fina a mitad de lote exigiría persistir por archivo antes de terminar, que hoy no hacemos.
Evidencia	Las métricas de la página 6 son columnas de la tabla <code>runs</code> , no un cálculo hecho para este documento. La cola de revisión es una consulta. El «qué cambió» entre normas es una comparación entre dos filas del historial.

4 Lo que no podemos demostrar

Los datos son sintéticos y no tenemos la referencia privada.

Podemos demostrar consistencia con norma y fuentes, no coincidencia con la clave del jurado.

La precisión de lectura sale de 45 documentos etiquetados por una persona.

Es una comprobación por muestreo; el 98,4 % debe leerse con ese tamaño delante.

Un solo nodo y un solo escritor.

Los ocho hilos sólo extraen; SQLite se escribe después, en serie. Para varios nodos habría que migrar el almacén.

El coste de visión es precio de lista.

La ejecución medida costó 0,00 \$ por caché y cuota gratuita. La fórmula sirve; el precio unitario es una estimación.

La recuperación es por lote, no por archivo.

La caché evita repetir visión, pero no hay cola durable por documento.

La automatización de escritorio necesita la consola abierta.

La carpeta vigilada y los lotes programados se disparan desde el navegador; no son un planificador de servidor.

5 La consola de revisión

La consola cubre el antes, el durante y el después del veredicto sin crear un segundo camino de decisión.

Entrada operativa.

Lotes manuales, programados o por carpeta vigilada; selección de formatos; progreso SSE; y reextracción explícita sin caché.

Revisión resoluble.

PDF y traza al lado, borrador para proveedor por correo, Gmail, Outlook o WhatsApp, y estado *pendiente* / *enviado* / *resuelto*.

Reglas y proveedores.

La política de veredictos vive en su pestaña; configuración controla fecha, tolerancia y formatos; el maestro fusionado se consulta en Proveedores.

Historial, enlace y comparación.

Cada factura y ejecución tiene URL; borrar una ejecución restaura la decisión anterior; Cambios explica qué se movió y por qué.

Indicadores accionables.

Euros por veredicto, duplicados evitados, motivos, proveedores problemáticos y cola pendiente, filtrables por lote.

De nada, Alberto. Ahora lo difícil será explicarle a tu jefe por qué antes tardabas tanto. Puedes dejar el Excel abierto para disimular.
